Roadmap: med-mal records platform

This is the meta-orchestration plan that sequences the work documented in `PLAN_med_mal_templates.md`, `PLAN_med_mal_extraction_pipeline.md`, and the v1.1 surfaces still in design. It exists so future contributors (and future-Claude sessions) can see the whole arc — not just the per-phase plans — and understand *why* the ordering is what it is, what's already shipped, and what hidden dependencies were surfaced from exploration that aren't in the underlying plans.

Context

`mike` is being shaped into a med-mal records platform: take a 1,500–3,000-page Epic PDF, scaffold the case in a project with the right folder taxonomy and review schemas, extract a citation-anchored event log defensible under cross-examination, run deterministic red-flag rules, and surface deadline tracking. The work spans two unbuilt plans (`PLAN_med_mal_templates.md` for the project-template scaffolding, `PLAN_med_mal_extraction_pipeline.md` for the structured-extraction pipeline) plus two v1.1 surfaces (templates ↔ extraction integration, deadline tracking) that compose them.

Validated against Minnesota primary sources in `RESEARCH_mn_med_mal_law.md` (Plutshack/Smith, § 145.682, § 548.251, § 144.293, Popovich, Dickhoff, Flom, Reinhardt, Cornfeldt). The validation pass produced ten substantive deltas to each underlying plan before any code shipped.

Why this order

Five phases, ordered to de-risk the biggest investment (extraction pipeline) by validating its design and exercising the schema/migration toolchain on a smaller change first:

- **Phase 0** is cheap insurance — the templates-plan validation surfaced ten deltas; the extraction-plan validation surfaced ten more, including red-flag rules that were 5-of-5 breach-only with no causation rule, no § 145.64 peer-review hard-refuse policy, no Plutshack `supports_element` tagging, and no MN-specific work-product / Rule 408 caveats. Going straight to code would have shipped all of those gaps into a multi-day build.
- **Phase 1** exercises the Supabase-branch + schema-migration toolchain on a one-column reversible change before Phase 2's three-table change. Mistakes are reversible. Phase 1 ships the user-facing template surface that scaffolds *future* projects; Phase 2 serves the active case.
- **Phase 2** is the multi-day main lift — the extraction pipeline that turns a 3,000-page Epic PDF into a citation-anchored event log.
- **Phase 3** is where templates and extraction *compose* — tabular reviews start populating from the event log rather than re-extracting per review.
- **Phase 4** caps the v1.1 surface the templates plan already gestured at — deadline tracking, now powered by auto-suggest from the event log.

Phase summary

#	Phase	Output	Est. effort	Status
0	Validate extraction-pipeline plan against MN-law research	"Plan deltas" appendix + Cited authority section in `PLAN_med_mal_extraction_pipeline.md`	1–2 hrs	✅ shipped (`3a97315`)
1	Ship templates feature	Working template picker + recommended-reviews strip; `projects.template_id` migration	~half day	✅ shipped (`73faac0`)
2	Build extraction pipeline	Event-log pipeline running against a real Epic PDF; bbox-anchored citations; 6 red-flag rules	multi-day	⏳ pending
3	Integrate templates ↔ extraction	"+ Medical Chronology" populates from `document_events` rather than re-extracting	~half day	⏳ pending
4	v1.1 deadline tracking	`projects.key_dates jsonb` + project-page widget with auto-suggest from event log	~half day	⏳ pending

Hidden dependencies surfaced during exploration

Surfaced from a read-through of the codebase before phasing began. These are NOT in the per-phase plans and are critical to the overall arc:

1. **`backend/migrations/` directory did not exist.** Phase 1 created it (`0001_projects_template_id.sql`) — the workflow is now established for Phase 2 and beyond.
2. **LLM provider layer does not support multimodal input.** `backend/src/lib/llm/claude.ts` and `gemini.ts` both accept text-only content. Phase 2 must extend the provider abstraction (signatures in `backend/src/lib/llm/index.ts`) for image/PDF parts. This is a sub-task hidden inside Phase 2's "page-by-page multimodal extraction" step — not a free addition to the multi-day estimate.
3. **No bbox-extraction primitive exists.** Existing PDF code (`backend/src/lib/convert.ts` + `pdfjs-dist` usage in `routes/projects.ts:754–765`) only counts pages and walks the outline. Phase 2 must build a per-page rendering + bbox-anchoring primitive. Decision point: pdfjs-dist text-layer-with-positions (cheap, works for typed text; fails on scanned/handwritten Epic content) vs. rasterize + LLM-returned bboxes (expensive but works on handwriting — the actual case for Epic ebooks).
4. **Supabase branching is gated on the Pro plan.** The current org (`rmmain-2176's projects`) is on the free plan; `create_branch` returns `PaymentRequiredException` . Until the plan is upgraded, schema migrations apply directly to prod via `mcp__supabase__apply_migration` . Documented in `CLAUDE.md` so future migrations don't burn round-trips on `create_branch` first. Prefer reversible changes (additive nullable columns with `if not exists`) so a bad migration can be rolled back without data loss.

Phase 0 — Validate extraction-pipeline plan ✅

Applied the same MN-law validation method that produced ten deltas on the templates plan to `PLAN_med_mal_extraction_pipeline.md` . Resulting deltas:

- Schema additions: `provider_role` , `episode_of_care` , `privacy_class` , `key_date_role` on `document_events` ; `supports_element` , `awaits_expert_affidavit` on `document_red_flags` ; restructured `medications jsonb` shape for the Mulder rule.

- Red-flag library rebalanced from 5 breach-only rules to 6 rules tagged across duty/breach/causation, including a new `temporal_anchor_causation` rule that surfaces tight anchors without asserting causation (Plutshack/Smith still require expert testimony for the ultimate finding).
- §Defenses extended with four MN-specific policies: § 145.64 peer-review hard-refuse (extraction halts entirely on peer-review-marked documents — the strictest policy in the codebase), § 144.293 mental-health redaction-by-default, Rule 408 settlement caveat (extracted-but-prefixed), Rule 26(b)(3) work-product opt-out.
- §Out of scope grew with two permanent architectural separations: causation-chain reasoning is reserved for the `builtin-causation-chain` tabular review (Plutshack/Smith expert testimony requirement); provider-defendant entity resolution is reserved for `builtin-provider-defendant-map` (Popovich two-factor test requires outside-the-record evidence).

Committed as `3a97315 docs: validate extraction-pipeline plan against MN-law research`. The deltas appendix + Cited authority section are inline at the end of `PLAN_med_mal_extraction_pipeline.md`.

Phase 1 — Ship templates feature ✅

Implemented `PLAN_med_mal_templates.md` end-to-end:

- **Schema:** `projects.template_id text null` (applied directly to prod after free-plan branching failed; folded into `schema.sql` + `backend/migrations/0001_projects_template_id.sql`).
- **Backend registry:** `backend/src/lib/templateIds.ts` (shared string-literal unions for `ProjectTemplateId` + `TabularSchemaId`) and `backend/src/lib/builtinProjectTemplates.ts` (one template `med-mal-case` with 12 top-level subfolders + nested `imaging`).
- **POST /projects:** parses optional `template_id`, validates against the registry (400 on unknown), batch-creates subfolders in two passes — Batch A inserts top-level folders and returns UUIDs; Batch B resolves the array-index parent refs to those UUIDs and inserts nested. Project deletion on Batch B failure makes the operation atomic from the user's perspective.
- **Frontend:** `frontend/src/app/components/tabular/{templateIds,builtinTabularSchemas}.ts` (mirror of backend; 11 schemas with ~120 column prompts tuned for Mayo/Epic ebook formatting); `frontend/src/app/components/projects/builtinProjectTemplates.ts` (frontend mirror, id + name + description + recommendedSchemaIds only — backend owns the subfolder list).
- **UI:** template dropdown in `NewProjectModal` between the CM number field and attribute pills; recommended-reviews strip on the project page reviews tab that renders only when `project.template_id` is set; `AddNewTRModal` extended with `initialTitle` and `initialColumnsConfig` props the strip uses to seed the modal.

Committed as `73faac0 feat(templates): med-mal-case project template`.

Phase 2 — Build extraction pipeline ⏳

Implements `PLAN_med_mal_extraction_pipeline.md` (post-Phase-0 revisions). Multi-day main lift.

Concrete first steps: 1. Supabase branch `extraction-pipeline` (or direct prod, if still on free plan) + apply the 3-table migration (`document_events`, `document_red_flags`, `document_extractions`). Smoke-test with `execute_sql` selects. 2. **Extend LLM provider layer for multimodal** — the hidden dependency. Start with Claude (Anthropic SDK has clean vision support); Gemini second. New module: `backend/src/lib/llm/multimodal.ts` wrapping per-provider image/PDF part formats. 3. **Build the bbox primitive** — new module:

`backend/src/lib/extraction/pdfPages.ts`. Test text-layer-with-positions first; fall back to rasterize + LLM-bbox for pages where text extraction yields zero text (handwritten / scanned). Cache rasters in R2 under `extractions/{userId}/{docId}/pages/{n}.png` per existing key conventions in `lib/storage.ts:150–188`. 4. `backend/src/lib/extraction/medMalExtractor.ts` — orchestrate per-page extraction calls; validate each event has `{source_page, source_bbox}` before persisting. Idempotent on `(document_id, extraction_run_id)`. Update `document_extractions.pages_complete` per page so the frontend can show progress. 5. `backend/src/lib/extraction/redFlagRules.ts` — six rules per the Phase-0 list. Each emits to `document_red_flags` with `supports_element` and `supporting_event_ids[]`. 6. `backend/src/routes/extraction.ts` — `POST /extraction/:documentId/run`, `GET /extraction/:documentId/events`, `GET /extraction/:documentId/red-flags`, `GET /extraction/:documentId/status`. SSE for the status poll if the existing chat-SSE pattern fits. 7. Frontend extraction view at `frontend/src/app/(pages)/projects/[id]/extraction/page.tsx` — side-by-side `DocView` + event timeline + red-flag list with bbox-click sync. 8. Integration-test on a real Epic PDF — expect 2–4 hours of multimodal calls; monitor cost (~$5–15 per 1K pages on Claude vision; ~$15–45 one-shot for a 3K-page PDF).

Risks tracked in `PLAN_med_mal_extraction_pipeline.md` §Risks accepted: cost, latency, bbox hallucination off-page (mitigated by validating bbox against page dimensions before persisting), v1 red-flag recall is narrow by design.

Phase 3 — Integrate templates ↔ extraction ⏳

Converts tabular reviews from "extract fresh per review" to "view over `document_events`" where an event log exists. Composes the two features into something neither alone delivers.

Per the architectural-tension note inline in `PLAN_med_mal_extraction_pipeline.md`: keep the event log narrow and authoritative; each tabular schema has its own extractor that *consults* the event log for date-anchoring but extracts schema-specific data independently. Cleanest separation of concerns.

Critical files: - `backend/src/routes/tabular.ts` — when a new review is created against a document with a completed `document_extractions` row, prefer event-log queries over fresh LLM extraction. - `frontend/src/app/components/tabular/AddNewTRModal.tsx` — banner "This document has an event log — review will populate from it" when applicable. - `frontend/src/app/components/projects/ProjectPage.tsx` — strip detects extracted-document state and adds a "Populate from event log" path.

Verification gate: opening "+ Medical Chronology" on a fully-extracted document takes < 2 s (no LLM call); same operation on a non-extracted document falls back to existing extraction.

Phase 4 — v1.1 deadline tracking ⏳

Finishes the v1.1 surface flagged in `PLAN_med_mal_templates.md`. With Phase 2's extraction pipeline pulling dates from records, the widget can auto-suggest from `document_events.key_date_role` rather than requiring manual entry.

Schema: add `projects.key_dates jsonb null` (12 raw date fields + 6 computed deadlines per `RESEARCH_mn_med_mal_law.md` § 4.2).

Backend: `backend/src/lib/keyDates.ts` for schema validation + deadline computation (`deadline_145682_id_affidavit = date_discovery_commenced_2604a + 180 days`, `deadline_4yr_sol = date_of_negligent_act + 4 years`, etc.). `PATCH /projects/:id` extends to accept `key_dates`.

Frontend: deadline-tracking widget on the project page (red < 120 days / yellow < 12 mo / green); `KeyDatesPanel.tsx` form for manual entry with auto-suggest from extracted events.

Verification: widget renders correct color for synthetic dates spanning all three bands; auto-suggest pre-fills `date_of_negligent_act` from the earliest `key_date_role='negligent_act_candidate'` event when available.

Risk register (cross-phase)

Risk	Mitigation	Phase
Stale assumptions in extraction-pipeline plan	Phase 0 validation pass against MN-law research	0 ✅
Frontend/backend mirror drift on schema IDs	`templateIds.ts` shared file pattern (two copies, intentional duplication)	1 ✅
Free-plan blocks `create_branch`	Direct prod migrations on reversible changes; document recipe in `CLAUDE.md`	1 ✅, 2, 4
LLM provider abstraction is text-only	Build `multimodal.ts` as a Phase 2 sub-task; do not assume vision support exists	2
Bbox hallucination off-page	Validate bbox against page dimensions before persisting	2
Multimodal cost on a 3K-page PDF	Budget-cap per run; resume-on-failure for partial extractions	2
Cloudflare Workers + Node-only APIs in multimodal modules	Test frontend build after each Phase 2 module lands	2
PHI leakage in extraction logs	Log only event ids, never raw page content (per `PLAN_med_mal_extraction_pipeline.md` §Defenses)	2
Architectural tension between event log and tabular schemas	Decided in Phase 0 deltas: event log stays narrow; per-schema extractors consult it	3

Out of scope (across all phases)

- Vector embeddings / GraphRAG / ColBERT — `RESEARCH_legal_rag.md` adjudicated structured extraction with bbox citations as the right choice over vectors for med-mal records.
- Cross-case search ("every shoulder-dystocia case in our firm") — explicitly out of `PLAN_med_mal_extraction_pipeline.md` scope.
- Automated demand-letter generation — Phase 5+ if ever; case-law territory.
- `legal:triage-nda` med-mal variant — separate skill-bundle build outside `mike/`; no plugin infrastructure currently installed at `~/.claude/plugins/`.
- Template versioning — accepted-risk in templates plan; existing projects are not retro-migrated when a template's subfolder list changes.
- Settlement/mediation folder, trial-exhibits folder — deferred to v1.1+ per `RESEARCH_mn_med_mal_law.md` § 1.3.
- **Causation-chain reasoning extraction** — permanent architectural separation per Phase 0 delta 9. Reserved for `builtin-causation-chain` tabular review or chat. Plutshack/Smith require expert testimony; Dickhoff loss-of-chance is fact-intensive.

- **Provider-defendant entity resolution** — permanent architectural separation per Phase 0 delta 9. Popovich two-factor test depends on outside-the-record evidence (advertising, intake forms, deposition testimony); the reliance prong cannot be satisfied from the chart alone (Rock v. Abdullah limit).

Verification gates per phase

Each phase ends with a review checkpoint before the next phase begins. Commit/PR pattern in use:

- Phase 0 → single commit `docs: validate extraction-pipeline plan against MN-law research` (`3a97315`).
- Phase 1 → single squashed commit `feat(templates): …` (`73faac0`).
- Phase 2 → multiple commits prefixed `feat(extraction): …`, split by module (schema, multimodal, extractor, rules, routes, UI).
- Phase 3 → single commit / PR `feat(integration): tabular reviews populate from event log`.
- Phase 4 → single commit / PR `feat(deadlines): v1.1 key-dates widget`.

Cross-cutting deliverables

- **`CLAUDE.md`** — updated after Phase 1 with the migrations workflow + Supabase free-plan branching caveat. Maintain on each schema change.
- **`.remember/now.md` + `today-*.md`** — session-level memory updated at the end of each phase so a resumed session can pick up cleanly. Local-only (not in repo).
- **PDF re-render** — every plan/research doc edit re-renders the companion `.pdf` via `/tmp/jenn-mike/md_to_pdf.py` (reconstructed with `markdown` + `weasyprint` after the original was wiped from `/tmp`). The script is local-only; if you don't have it, any markdown→PDF tool works.